Lab # 06

Experiment No 06

Implementation of Constructors

Objective

An introduction with Constructors and Destructors

Software Tools

1. Dev++

Constructors

A constructor in C++ is a **special method** that is automatically called when an object of a class is created.

To create a constructor, use the same name as the class, followed by parentheses `()`:

Example 1

```
class MyClass {     // The class
  public:           // Access specifier
MyClass() {     // Constructor
cout << "Hello World!";
    }
};  int
main() {
  MyClass myObj;    // Create an object of MyClass (this will call the
constructor)   return 0;
}
```

Note: The constructor has the same name as the class, it is `public`, and always does not have any return value. it

Constructor Parameters

Constructors can also take parameters (just like regular functions), which can be useful for setting initial values for attributes.

The following class have `brand`, `model` and `year` attributes, and a constructor with different parameters. Inside the constructor we set the attributes equal to the constructor parameters (`brand=x`, etc). When we call the constructor (by creating an object of the class), we pass parameters to the constructor, which will set the value of the corresponding attributes to the same:

Example 2

```
class Car {         // The class
  public:           // Access specifier
string brand;  // Attribute     string
model;  // Attribute     int year;
// Attribute

    Car(string x, string y, int z) { // Constructor with parameters
brand = x;       model = y;       year = z;

    }
};  int
main() {
  // Create Car objects and call the constructor with different values
  Car carObj1("BMW", "X5", 1999);
  Car carObj2("Ford", "Mustang", 1969);

  // Print values
  cout << carObj1.brand << " " << carObj1.model << " " <<
carObj1.year << "\n";   cout << carObj2.brand << " " <<
carObj2.model << " " << carObj2.year << "\n";   return 0;
}
```

Just like functions, constructors can also be defined outside the class. First, declare the constructor inside the class, and then define it outside of the class by specifying the name of the class, followed by the scope resolution `::` operator, followed by the name of the constructor (which is the same as the class):

Example 3

```
class Car {        // The class
  public:          // Access specifier
string brand;  // Attribute     string
model;  // Attribute     int year;
// Attribute
    Car(string x, string y, int z); // Constructor declaration
};

// Constructor definition outside the class
Car::Car(string x, string y, int z) {
brand = x;   model = y;   year = z;
}  int main()
{
  // Create Car objects and call the constructor with different values
  Car carObj1("BMW", "X5", 1999);
  Car carObj2("Ford", "Mustang", 1969);

  // Print values   cout << carObj1.brand << " " <<
carObj1.model << " " << carObj1.year << "\n";   cout <<
carObj2.brand << " " << carObj2.model << " " <<
carObj2.year << "\n";   return 0;
}
```

C++ Class Example: Store and Display Employee Information

Let's see another example of C++ class where we are storing and displaying employee information using method.

Example 4

```
1. #include <iostream>
2.          using namespace std;
3.          class Employee {
4.          public:
5.          int id;//data member (also instance variable)
6.          string name;//data member(also instance variable)
7.          float salary;
8.          void insert(int i, string n, float s)
9.          {
10.         id = i;
11.         name = n;
12.         salary = s;
13.         }
14.         void display()
15.         {
16.         cout<<id<<" "<<name<<" "<<salary<<endl;
17.         }
18.         };
19.         int main(void) {
20.         Employee e1; //creating an object of Employee
21.         Employee e2; //creating an object of Employee
```

```
22.         e1.insert(201, "Sonoo",990000);
23.         e2.insert(202, "Nakul", 29000);
24.         e1.display();
25.         e2.display();
26.         return 0;
27.         }
```

Output:

```
201   Sonoo   990000
202   Nakul   29000
```

C++ Default Constructor

A constructor which has no argument is known as default constructor. It is invoked at the time of creating object.

Let's see the simple example of C++ default Constructor.

Example 5

```
1. #include <iostream>
2.         using namespace std;
3.         class Employee
4.         {
5.         public:
6.         Employee()
7.         {
8.           cout<<"Default Constructor Invoked"<<endl;
9.         }
10.        };
11.        int main(void)
```

```
12.     {
13.     Employee e1; //creating an object of Employee
14.     Employee e2;
15.     return 0;
16.     }
```

Output:

```
Default Constructor Invoked
Default Constructor Invoked
```

C++ Parameterized Constructor

A constructor which has parameters is called parameterized constructor. It is used to provide different values to distinct objects.

Let's see the simple example of C++ Parameterized Constructor.

Example 6

```
#include <iostream>
using namespace std;
class Employee {
public:
       int id;//data member (also instance variable)
string name;//data member(also instance variable)        float
salary;
       Employee(int i, string n, float s)
        {
id = i;
name = n;
salary = s;
        }
       void display()
        {
            cout<<id<<"  "<<name<<"  "<<salary<<endl;
        }
};
int main(void) {
```

```
    Employee e1 =Employee(101, "Sonoo", 890000); //creating an object of
Employee
    Employee e2=Employee(102, "Nakul", 59000);
e1.display();       e2.display();       return 0;
}
```

Output:

```
101   Sonoo   890000
102   Nakul   59000
```

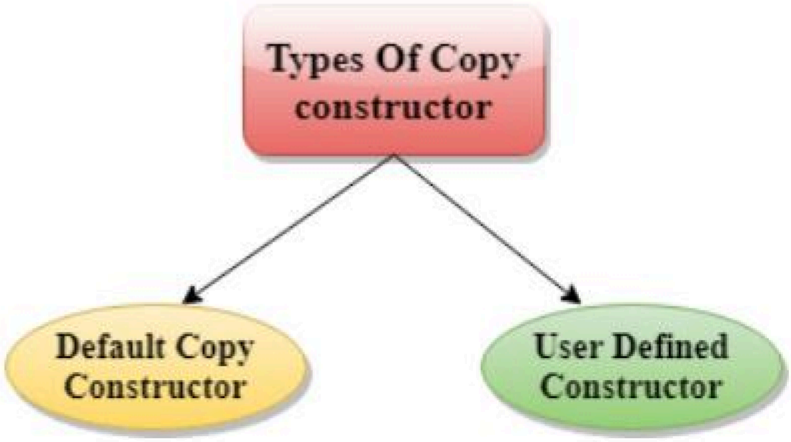
C++ Copy Constructor

A Copy constructor is an **overloaded** constructor used to declare and initialize an object from another object.

Copy Constructor is of two types:

- **Default Copy constructor:** The compiler defines the default copy constructor. If the user defines no copy constructor, compiler supplies its constructor.
- **User Defined constructor:** The programmer defines the user-defined constructor.

Lab # 06



Syntax Of User-defined Copy Constructor:

1. Class_name(**const** class_name &old_object);

Consider the following situation:

```
1.  class A
2.  {
3.  A(A &x) //  copy constructor.
4.  {
5.  // copyconstructor.
6.  }
7.  }
```

In the above case, **copy constructor can be called in the following ways:**

- **A a2(a1);**
- **A a2 = a1;**

} **a1 initialises the a2 object.**

Lab # 06

Let's see a simple example of the copy constructor.

// program of the copy constructor.

Example 7

```
1. #include <iostream>
2.         using namespace std;
3.         class A
4.         {
5.         public:
6.         int x;
7.         A(int a)              // parameterized constructor.
8.         {
9.         x=a;
10.        }
11.        A(A &i)             // copy constructor
12.        {
13.        x = i.x;
14.        }
15.        };
16.        int main()
17.        {
18.        A a1(20);             // Calling the parameterized constructor.
19.        A a2(a1);              //  Calling the copy constructor.
20.        cout<<a2.x;
21.        return 0;
22.        }
```

Output:

20

When Copy Constructor is called

Copy Constructor is called in the following scenarios:

- When we initialize the object with another existing object of the same class type. For example, Student s1 = s2, where Student is the class.
- When the object of the same class type is passed by value as an argument.
  - When the function returns the object of the same class type by value.

C++ Destructor

A destructor works opposite to constructor; it destructs the objects of classes. It can be defined only once in a class. Like constructors, it is invoked automatically.

A destructor is defined like constructor. It must have same name as class. But it is prefixed with a tilde sign (~).

Note: C++ destructor cannot have parameters. Moreover, modifiers can't be applied on destructors.

C++ Constructor and Destructor Example

Let's see an example of constructor and destructor in C++ which is called automatically.

Example 8

```
1. #include <iostream>
2.         using namespace std;
```

```
3.  class Employee
4.  {
5.  public:
6.  Employee()
7.  {
8.    cout<<"Constructor Invoked"<<endl;
9.  }
10. ~Employee()
11. {
12.    cout<<"Destructor Invoked"<<endl;
13. }
14. };
15. int main(void)
16. {
17. Employee e1; //creating an object of Employee
18. Employee e2; //creating an object of Employee
19. return 0;
20. }
```

Output:

```
Constructor Invoked
Constructor Invoked
Destructor Invoked
Destructor Invoked
```

C++ this Pointer

In C++ programming, **this** is a keyword that refers to the current instance of the class. There can be 3 main usage of this keyword in C++.

- It can be used **to pass current object as a parameter to another method.**
- It can be used **to refer current class instance variable.**
- It can be used **to declare indexers.**

C++ this Pointer Example

Let's see the example of this keyword in C++ that refers to the fields of current class.

Example 9

```
1. #include <iostream>
2.          using namespace std;
3.          class Employee {
4.          public:
5.          int id; //data member (also instance variable)
6.          string name; //data member(also instance variable)
7.          float salary;
8.          Employee(int id, string name, float salary)
9.          {
10.         this->id = id;
11.         this->name = name;
12.         this->salary = salary;
13.         }
14.         void display()
15.         {
16.         cout<<id<<"  "<<name<<"  "<<salary<<endl;
17.         }
18.         };
19.         int main(void) {
```

```
20.     Employee e1 =Employee(101, "Sonoo", 890000); //creating an object of Employee
21.     Employee e2=Employee(102, "Nakul", 59000); //creating an object of Employee
22.     e1.display();
23.     e2.display();
24.     return 0;
25.     }
```

Output:

```
101   Sonoo   890000
102   Nakul   59000
```

Task No 1:

- **Define a class Customer that holds private fields for a customer's ID, first name, last name, address and credit limit.**
- **Define functions that set the fields of customer. For example setCustomerID().**
- **Define functions that show the fields of customer. For example getCustomerID().**
- **Use constructor to set the default values to each of the field.**
- **Overload at least six constructor of the customer class.**
- **Define a function that takes input from user and set the all fields of customer data.**

Define a function that displays the entire customer's data.

Lab Performance Evaluation:
Read and practice the manual. Performance is evaluated at the end of lab based on successfully running and understanding of examples, tasks and viva using defined rubrics.